STRIDE Threat Model

Enterprise DevSecOps Security Lab - detailed threat analysis using Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.

Project	Enterprise DevSecOps Security Lab
Document owner	Jagriti Banerjee
Document type	STRIDE threat model
Scope	Application, GitHub, CI/CD, registry, Kubernetes, ArgoCD, policy-as-code, runtime detection and monitoring
Modeling approach	Component-based STRIDE analysis, attack paths, impact, existing controls, validation evidence and remediation guidance

Classification: Private lab documentation / portfolio evidence. This document is designed for security review, recruiter evaluation, and technical audit walkthroughs.

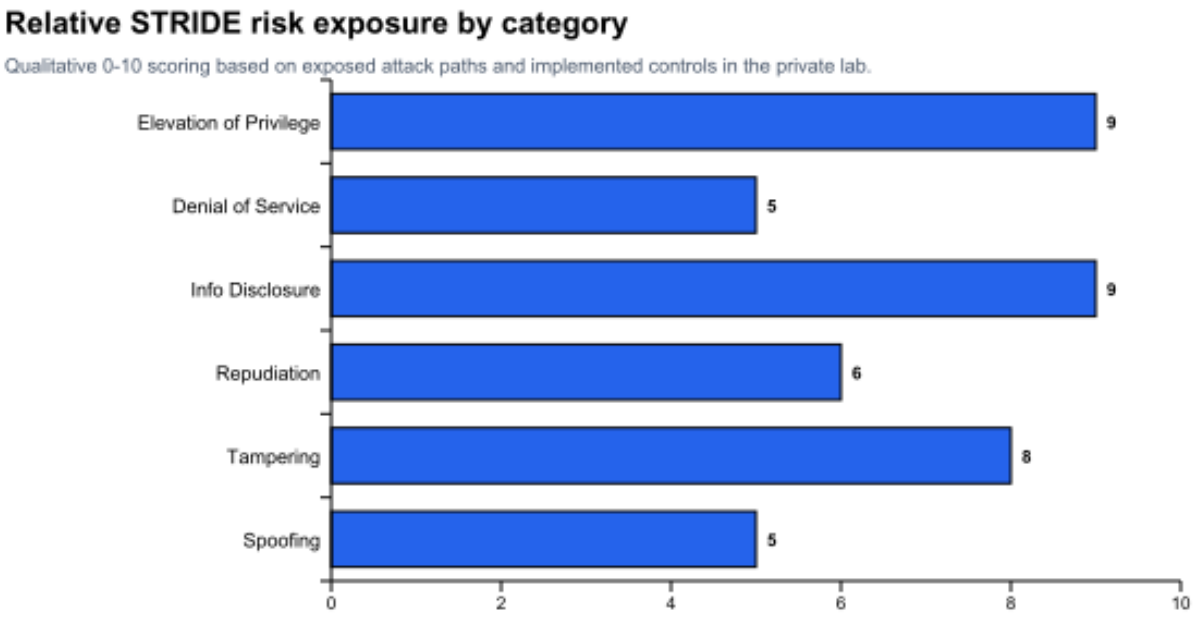
Document contents

Section group	What the reviewer will find
Architecture and scope	System overview, trust boundaries, assets and data-flow diagram.
Threat analysis	STRIDE categories, attack paths, affected components and residual risk.
Controls and validation	Preventive, detective, corrective and governance controls with evidence mapping.
Action plan	Risk treatment, retest criteria and enterprise hardening recommendations.

1. STRIDE Methodology

STRIDE is used to classify threats across six categories. The purpose of this document is to map each category to concrete threats in the Enterprise DevSecOps lab, identify affected components, describe trigger conditions, document existing controls and highlight residual risk.

STRIDE category	Security property challenged	Typical question asked during review
Spoofing	Authentication / identity	Can an attacker pretend to be a trusted user, workflow, service account, image or repository?
Tampering	Integrity	Can an attacker modify code, images, manifests, logs, SBOMs or runtime state without detection?
Repudiation	Accountability	Can an actor deny making a change because evidence, logs or audit trails are missing?
Information Disclosure	Confidentiality	Can sensitive data, secrets, package details, database rows or internal system information be exposed?
Denial of Service	Availability	Can a component be exhausted, crashed, blocked or forced into restart loops?
Elevation of Privilege	Authorization / isolation	Can an attacker gain permissions beyond the intended role or container boundary?



2. STRIDE by Component Matrix

The matrix below links major components to STRIDE categories. This helps reviewers see which threats apply to each part of the system and which controls should be validated first.

Component	Spoofing	Tampering	Repudiation	Info Disclosure	DoS	Elevation of Privilege
Flask app	Fake client requests with malicious parameters.	Manipulate query input or shell command parameters.	Insufficient application audit trail.	SQLi reveals records; errors reveal internals.	Abuse expensive endpoints or request floods.	Command injection may execute container commands.
GitHub repo	Attacker uses stolen account/token.	Workflow/Dockerfile/ manifests modified.	Weak commit attribution.	Secrets accidentally committed.	Blocking branch or workflow abuse.	Workflow token used beyond intended scope.
GitHub Actions	Malicious third-party action impersonates trusted action.	Build artifact changed after scan.	Incomplete workflow logs/artifacts.	Secrets printed in logs.	Runner exhaustion or dependency failure.	Overbroad GITHUB_TOKEN permissions.
GHCR registry	Fake image or tag impersonates approved artifact.	Image tag replaced.	Lack of digest audit trail.	Package metadata exposes internals.	Registry unavailable.	Unauthorized package write access.
ArgoCD	Fake repo credential or unauthorized UI/API user.	Application spec modified.	Manual sync without clear approval.	Repo secret exposure.	Sync storms or broken desired state.	Controller privileges abused.
Kubernetes runtime	Spoofed service account/pod identity.	Manifest drift or hostPath tampering.	Weak audit evidence.	Secret read, filesystem disclosure.	Resource exhaustion.	Privileged pod or cluster-admin SA.
Monitoring	Fake alerts or spoofed metrics.	Dashboard/rule tampering.	Missing evidence for alerts.	Sensitive alert fields exposed.	Metric/alert pipeline outage.	Admin Grafana/Prometheus access abuse.

3. Spoofing Threats

Spoofing threats occur when an attacker impersonates a trusted identity such as a developer, GitHub workflow, service account, image, ArgoCD source repository or monitoring source.

ID	Threat	Scenario	Severity	Primary controls	Validation
ST-S-01	Developer or maintainer account spoofing	A stolen account pushes malicious code or changes workflow security gates.	High	Branch protection, PR review, least-privilege workflow permissions, Scorecard review.	Check GitHub audit trail, workflow actor, commit metadata and branch protection status.
ST-S-02	Container image spoofing	A tag such as latest points to a different image than the scanned artifact.	High	Image digest capture, Cosign signing, SBOM/provenance attestation.	Verify digest using docker buildx imagetools inspect and cosign verify.
ST-S-03	ServiceAccount identity misuse	Compromised pod token impersonates a Kubernetes identity.	High	Automount disabled, dedicated SA, RBAC least privilege.	Run kubectl auth can-i as the service account.

Triage checklist

Step	Action
1	Identify the affected component, namespace, workload, workflow run, image digest or log source.
2	Confirm whether the behavior is expected lab activity, authorized testing, or unauthorized change.
3	Collect evidence: command output, screenshot, workflow artifact, log entry, digest, SBOM/provenance or Prometheus query result.
4	Determine blast radius: repository, registry, cluster namespace, service account, workload, runtime node or monitoring stack.
5	Apply containment: disable token, revert commit, delete workload, block policy, scale down component, or isolate namespace.

Response and evidence expectations

Response area	Required action	Evidence artifact
Containment	Stop the affected workflow, workload, token, sync operation or namespace path.	kubectl output, GitHub workflow result, ArgoCD status or incident notes.
Eradication	Remove the insecure configuration, vulnerable input pattern, overprivileged binding or unsafe artifact.	Diff, remediation commit, updated manifest, secure code example.
Recovery	Re-run security gates, redeploy through GitOps and confirm monitoring returns to normal.	Retest proof, green pipeline, Prometheus/Grafana screenshot.
Lessons learned	Update threat model, runbook, exception register and control validation matrix.	Updated PDF/MD artifact and evidence index entry.

4. Tampering Threats

Tampering threats involve unauthorized modification of source code, dependencies, container images, Kubernetes manifests, policies, logs, dashboards, or evidence.

ID	Threat	Scenario	Severity	Primary controls	Validation
ST-T-01	Workflow tampering	An attacker modifies security-gates.yml to disable failures.	Critical	Required PR review, branch protection, CODEOWNERS, workflow diff review.	Compare workflow diff and check required status checks.
ST-T-02	Manifest tampering	Attacker removes readOnlyRootFilesystem, seccomp or resource limits.	High	Trivy config scan, Gatekeeper constraints, ArgoCD self-heal.	Run trivy config and kubectl get deployment securityContext.
ST-T-03	SBOM/provenance tampering	Artifact metadata is edited after build.	High	Cosign attestation and signature verification.	Run cosign verify-attestation and compare digest.

Triage checklist

Step	Action
1	Identify the affected component, namespace, workload, workflow run, image digest or log source.
2	Confirm whether the behavior is expected lab activity, authorized testing, or unauthorized change.
3	Collect evidence: command output, screenshot, workflow artifact, log entry, digest, SBOM/provenance or Prometheus query result.
4	Determine blast radius: repository, registry, cluster namespace, service account, workload, runtime node or monitoring stack.
5	Apply containment: disable token, revert commit, delete workload, block policy, scale down component, or isolate namespace.

Response and evidence expectations

Response area	Required action	Evidence artifact
Containment	Stop the affected workflow, workload, token, sync operation or namespace path.	kubectl output, GitHub workflow result, ArgoCD status or incident notes.
Eradication	Remove the insecure configuration, vulnerable input pattern, overprivileged binding or unsafe artifact.	Diff, remediation commit, updated manifest, secure code example.
Recovery	Re-run security gates, redeploy through GitOps and confirm monitoring returns to normal.	Retest proof, green pipeline, Prometheus/Grafana screenshot.
Lessons learned	Update threat model, runbook, exception register and control validation matrix.	Updated PDF/MD artifact and evidence index entry.

5. Repudiation Threats

Repudiation threats occur when an actor can deny performing an action because logging, workflow evidence, audit trails, or report retention are insufficient.

ID	Threat	Scenario	Severity	Primary controls	Validation
ST-R-01	Manual cluster change without evidence	A live resource is changed with kubectl and later denied.	Medium	ArgoCD drift detection, self-heal, event capture.	Review ArgoCD app history and Kubernetes events.
ST-R-02	Security gate bypass not recorded	A failing check is ignored without documented exception.	High	Vulnerability exceptions, VEX notes, gate summary artifact.	Confirm exception owner, expiry, and risk acceptance.
ST-R-03	Incident response actions not logged	Containment or remediation steps are not captured.	Medium	IR runbook, evidence index, screenshots, reports.	Match timeline to evidence screenshots and command outputs.

Triage checklist

Step	Action
1	Identify the affected component, namespace, workload, workflow run, image digest or log source.
2	Confirm whether the behavior is expected lab activity, authorized testing, or unauthorized change.
3	Collect evidence: command output, screenshot, workflow artifact, log entry, digest, SBOM/provenance or Prometheus query result.
4	Determine blast radius: repository, registry, cluster namespace, service account, workload, runtime node or monitoring stack.
5	Apply containment: disable token, revert commit, delete workload, block policy, scale down component, or isolate namespace.

Response and evidence expectations

Response area	Required action	Evidence artifact
Containment	Stop the affected workflow, workload, token, sync operation or namespace path.	kubectl output, GitHub workflow result, ArgoCD status or incident notes.
Eradication	Remove the insecure configuration, vulnerable input pattern, overprivileged binding or unsafe artifact.	Diff, remediation commit, updated manifest, secure code example.
Recovery	Re-run security gates, redeploy through GitOps and confirm monitoring returns to normal.	Retest proof, green pipeline, Prometheus/Grafana screenshot.
Lessons learned	Update threat model, runbook, exception register and control validation matrix.	Updated PDF/MD artifact and evidence index entry.

6. Information Disclosure Threats

Information disclosure threats expose secrets, database rows, package details, runtime filesystem content, Kubernetes objects, or internal security reports to unauthorized parties.

ID	Threat	Scenario	Severity	Primary controls	Validation
ST-I-01	SQL injection data exposure	The /user route returns more records than intended.	High	Parameterized SQL, input validation, tests, SAST.	Run safe and injection requests; confirm only expected row is returned after fix.
ST-I-02	Secret exposure in repo or logs	Tokens appear in Git history or workflow output.	Critical	Trivy secret scan, restricted permissions, no plaintext tokens.	Review Trivy secret scan and workflow logs.
ST-I-03	Host filesystem disclosure	Privileged hostPath pod reads node files.	Critical	Pod Security restricted, Gatekeeper, Falco detection.	Attempt privileged pod creation and confirm Forbidden result.

Triage checklist

Step	Action
1	Identify the affected component, namespace, workload, workflow run, image digest or log source.
2	Confirm whether the behavior is expected lab activity, authorized testing, or unauthorized change.
3	Collect evidence: command output, screenshot, workflow artifact, log entry, digest, SBOM/provenance or Prometheus query result.
4	Determine blast radius: repository, registry, cluster namespace, service account, workload, runtime node or monitoring stack.
5	Apply containment: disable token, revert commit, delete workload, block policy, scale down component, or isolate namespace.

Response and evidence expectations

Response area	Required action	Evidence artifact
Containment	Stop the affected workflow, workload, token, sync operation or namespace path.	kubectl output, GitHub workflow result, ArgoCD status or incident notes.
Eradication	Remove the insecure configuration, vulnerable input pattern, overprivileged binding or unsafe artifact.	Diff, remediation commit, updated manifest, secure code example.
Recovery	Re-run security gates, redeploy through GitOps and confirm monitoring returns to normal.	Retest proof, green pipeline, Prometheus/Grafana screenshot.
Lessons learned	Update threat model, runbook, exception register and control validation matrix.	Updated PDF/MD artifact and evidence index entry.

7. Denial of Service Threats

Denial of Service threats reduce availability by exhausting resources, breaking deployment pipelines, preventing ArgoCD sync, or overwhelming monitoring components.

ID	Threat	Scenario	Severity	Primary controls	Validation
ST-D-01	Application resource exhaustion	Repeated requests consume CPU/memory and restart the pod.	Medium	Resource requests/limits, liveness/readiness probes.	Check pod restarts and resource usage.
ST-D-02	CI/CD pipeline blockage	Dependency service outage or slow scan prevents build completion.	Medium	Artifact retention, scheduled scans, retry policy.	Review workflow duration and failure reason.
ST-D-03	Monitoring pipeline overload	Falco or Prometheus drops events.	High	Falco dropped-event alerts, Prometheus rules.	Query dropped events and alert rule state.

Triage checklist

Step	Action
1	Identify the affected component, namespace, workload, workflow run, image digest or log source.
2	Confirm whether the behavior is expected lab activity, authorized testing, or unauthorized change.
3	Collect evidence: command output, screenshot, workflow artifact, log entry, digest, SBOM/provenance or Prometheus query result.
4	Determine blast radius: repository, registry, cluster namespace, service account, workload, runtime node or monitoring stack.
5	Apply containment: disable token, revert commit, delete workload, block policy, scale down component, or isolate namespace.

Response and evidence expectations

Response area	Required action	Evidence artifact
Containment	Stop the affected workflow, workload, token, sync operation or namespace path.	kubectl output, GitHub workflow result, ArgoCD status or incident notes.
Eradication	Remove the insecure configuration, vulnerable input pattern, overprivileged binding or unsafe artifact.	Diff, remediation commit, updated manifest, secure code example.
Recovery	Re-run security gates, redeploy through GitOps and confirm monitoring returns to normal.	Retest proof, green pipeline, Prometheus/Grafana screenshot.
Lessons learned	Update threat model, runbook, exception register and control validation matrix.	Updated PDF/MD artifact and evidence index entry.

8. Elevation of Privilege Threats

Elevation of Privilege threats allow an attacker to gain higher permissions than intended, such as cluster-admin, privileged container access, host filesystem access, or workflow write access.

ID	Threat	Scenario	Severity	Primary controls	Validation
ST-E-01	Cluster-admin ServiceAccount binding	Overprivileged SA can read secrets and create clusterrolebindings.	Critical	Delete ClusterRoleBinding, namespace RoleBinding only.	Run kubectl auth can-i get secrets before and after fix.
ST-E-02	Privileged pod container escape path	Pod runs privileged and mounts host root.	Critical	Pod Security restricted, admission denial, Falco alert.	Apply bad pod and confirm it is blocked.
ST-E-03	Workflow token privilege escalation	Workflow token can write packages or code unexpectedly.	High	Explicit permissions block, least privilege, Scorecard.	Inspect workflow permissions and GitHub settings.

Triage checklist

Step	Action
1	Identify the affected component, namespace, workload, workflow run, image digest or log source.
2	Confirm whether the behavior is expected lab activity, authorized testing, or unauthorized change.
3	Collect evidence: command output, screenshot, workflow artifact, log entry, digest, SBOM/provenance or Prometheus query result.
4	Determine blast radius: repository, registry, cluster namespace, service account, workload, runtime node or monitoring stack.
5	Apply containment: disable token, revert commit, delete workload, block policy, scale down component, or isolate namespace.

Response and evidence expectations

Response area	Required action	Evidence artifact
Containment	Stop the affected workflow, workload, token, sync operation or namespace path.	kubectl output, GitHub workflow result, ArgoCD status or incident notes.
Eradication	Remove the insecure configuration, vulnerable input pattern, overprivileged binding or unsafe artifact.	Diff, remediation commit, updated manifest, secure code example.
Recovery	Re-run security gates, redeploy through GitOps and confirm monitoring returns to normal.	Retest proof, green pipeline, Prometheus/Grafana screenshot.
Lessons learned	Update threat model, runbook, exception register and control validation matrix.	Updated PDF/MD artifact and evidence index entry.

9. Mitigation Roadmap

Phase	Mitigation	STRIDE categories reduced	Expected evidence
Immediate	Keep security-gates.yml required on pull requests and preserve artifacts.	Tampering, repudiation, information disclosure.	Workflow run, gate summary, SARIF uploads.
Immediate	Maintain RBAC least privilege and Pod Security restricted labels.	Spoofing, elevation of privilege, information disclosure.	kubectl auth can-i results, Forbidden privileged pod proof.
Near-term	Use digest-pinned images and admission-time signature verification.	Spoofing, tampering.	Kubernetes manifest digest, cosign verify, policy enforcement result.
Near-term	Generate SLSA provenance in GitHub Actions rather than local VM only.	Tampering, repudiation.	Trusted builder provenance attestation.
Near-term	Move vulnerable routes to isolated lab branch and keep main branch passing tests.	Information disclosure, elevation of privilege.	Retest proof and green security gates.
Long-term	Integrate centralized audit logging and SIEM forwarding.	Repudiation, tampering, information disclosure.	Audit log retention and alert routing evidence.

10. STRIDE Retest Criteria

Category	Pass condition
Spoofing	All deployable images can be tied to a known digest and verified signature. Service accounts cannot impersonate cluster-admin capabilities.
Tampering	Code, workflow, image, SBOM and manifest changes are detected through PR diff, security gates, Cosign, Trivy and ArgoCD.
Repudiation	Every remediation and exception has evidence, owner, rationale, timestamped workflow or screenshot artifact.
Information Disclosure	Injection tests are blocked or remediated; secrets are not found in repo or logs; pods cannot read Kubernetes secrets unless explicitly allowed.
Denial of Service	Workloads have resource limits, health probes, and monitoring queries for restarts/drops.
Elevation of Privilege	Privileged pod creation is blocked; RBAC can-i tests prove no secret or cluster-admin access for app SAs.